COER UNIVERSITY, ROORKEE

Department of Computer Science & Engineering
Full Stack Development

Mini E-Commerce Project
Lab Sheet 3 — Answer / Submission File

Building the Storefront with React (App.jsx + App.css)

Student Name	ABHIGYAN DUBEY
Program	B.Tech CSE, 5th Semester
CU ID	CU24250059
Course Code	BTCS303P
Assignment	AS01 — Project Phase 3 (React)
Project Theme	OtakuVerse — Anime Merchandise & Webnovel Store
Repository	GitHub: https://github.com/cu24250059-pixel/Full_stack_BTCS303P
Files Submitted	App.jsx, App.css
Status	Completed — screenshots attached as proof of implementation

This document contains the completed Lab Sheet 3 source code (React components and styling) along with live screenshots of the working storefront as submitted proof.

Requirement Checklist

[OK] Product data array with id, name, price and image for each product
[OK] ProductCard and ProductList components rendering the product grid with Add to Cart buttons
[OK] Cart state managed with useState; quantity increases when the same product is added again
[OK] Cart component showing item name, unit price × quantity, subtotal, Remove button and Grand Total
[OK] Empty-cart message displayed when no items are present
[OK] CheckoutForm with Name, Email, Phone, Address and Payment Method fields and form validation
[OK] Navbar with live cart count badge; footer with copyright text
[OK] App.css with responsive CSS Grid product list, card hover effect and a mobile media query

1. JavaScript Code — App.jsx

```
import React, { useState } from "react";
import "./App.css";

const products = [
  { id: 1, name: "Wireless Headphones", price: 1499,
    image: "https://images.unsplash.com/photo-1505740420928-5e560c06d30e?w=500" },
  { id: 2, name: "Smart Watch", price: 2499,
    image: "https://images.unsplash.com/photo-1523275335684-37898b6baf30?w=500" },
  { id: 3, name: "Running Shoes", price: 1999,
    image: "https://images.unsplash.com/photo-1542291026-7eec264c27ff?w=500" },
  { id: 4, name: "Backpack", price: 999,
    image: "https://images.unsplash.com/photo-1553062407-98eeb64c6a62?w=500" },
  { id: 5, name: "Sunglasses", price: 799,
    image: "https://images.unsplash.com/photo-1511499767150-a48a237f0083?w=500" },
  { id: 6, name: "Smartphone", price: 14999,
    image: "https://images.unsplash.com/photo-1511707171634-5f897ff02aa9?w=500" },
];

function ProductCard({ product, addToCart }) {
  return (
    <div className="product-card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <p className="price">■{product.price}</p>
      <button onClick={() => addToCart(product)}>Add to Cart</button>
    </div>
  );
}

function ProductList({ addToCart }) {
  return (
    <div className="product-list">
      {products.map((product) => (
        <ProductCard key={product.id} product={product} addToCart={addToCart} />
      ))}
    </div>
  );
}

function Cart({ cart, removeFromCart }) {
  if (cart.length === 0) {
    return (
      <div className="cart">
        <h2>Shopping Cart</h2>
        <p>Your cart is empty.</p>
      </div>
    );
  }

  const grandTotal = cart.reduce(
    (total, item) => total + item.product.price * item.quantity,
    0
  );

  return (
    <div className="cart">
      <h2>Shopping Cart</h2>

      {cart.map((item) => {
        const subtotal = item.product.price * item.quantity;

        return (
```

```
          <div className="cart-item" key={item.product.id}>
            <div>
              <h3>{item.product.name}</h3>
              <p>■{item.product.price} × {item.quantity}</p>
              <p>Subtotal: ■{subtotal}</p>
            </div>

            <button
              className="remove-btn"
              onClick={() => removeFromCart(item.product.id)}
            >
              Remove
            </button>
          </div>
        );
      })}

      <h2 className="grand-total">Grand Total: ■{grandTotal}</h2>
    </div>
  );
}

function CheckoutForm({ clearCart }) {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [phone, setPhone] = useState("");
  const [address, setAddress] = useState("");
  const [payment, setPayment] = useState("Cash on Delivery");

  const handleSubmit = (e) => {
    e.preventDefault();

    if (!name || !email || !phone || !address) {
      alert("Please fill all the fields.");
      return;
    }

    alert(`Order placed successfully!\n\nThank you ${name}!`);

    clearCart();
    setName("");
    setEmail("");
    setPhone("");
    setAddress("");
    setPayment("Cash on Delivery");
  };

  return (
    <div className="checkout">
      <h2>Checkout</h2>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Enter your name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />

        <input
          type="email"
          placeholder="Enter your email"
```

```
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />

        <input
          type="tel"
          placeholder="Enter your phone number"
          value={phone}
          onChange={(e) => setPhone(e.target.value)}
        />

        <textarea
          placeholder="Enter your address"
          value={address}
          onChange={(e) => setAddress(e.target.value)}
        />

        <select value={payment} onChange={(e) => setPayment(e.target.value)}>
          <option>Cash on Delivery</option>
          <option>UPI</option>
          <option>Credit Card</option>
          <option>Debit Card</option>
        </select>

        <button type="submit">Place Order</button>
      </form>
    </div>
  );
}

function App() {
  const [cart, setCart] = useState([]);
  const [showCart, setShowCart] = useState(false);

  const addToCart = (product) => {
    setCart((currentCart) => {
      const existingProduct = currentCart.find(
        (item) => item.product.id === product.id
      );

      if (existingProduct) {
        return currentCart.map((item) =>
          item.product.id === product.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      }

      return [...currentCart, { product, quantity: 1 }];
    });

    alert(`${product.name} added to cart!`);
  };

  const removeFromCart = (productId) => {
    setCart((currentCart) =>
      currentCart.filter((item) => item.product.id !== productId)
    );
  };

  const clearCart = () => setCart([]);
```

```
  const cartCount = cart.reduce(
    (total, item) => total + item.quantity,
    0
  );

  return (
    <div className="app">
      <nav className="navbar">
        <h1>MyStore</h1>

        <button
          className="cart-button"
          onClick={() => setShowCart(!showCart)}
        >
          Cart <span>{cartCount}</span>
        </button>
      </nav>

      <main>
        <h2 className="title">Our Products</h2>

        <ProductList addToCart={addToCart} />

        {showCart && (
          <Cart cart={cart} removeFromCart={removeFromCart} />
        )}

        {cart.length > 0 && (
          <CheckoutForm clearCart={clearCart} />
        )}
      </main>

      <footer>
        <p>© 2026 MyStore. All Rights Reserved.</p>
      </footer>
    </div>
  );
}

export default App;
```

2. CSS Code — App.css

```
* { margin: 0; padding: 0; box-sizing: border-box; }

body { font-family: Arial, sans-serif; background: #f5f5f5; color: #222; }

.navbar {
  background: #222; color: white; padding: 18px 50px;
  display: flex; justify-content: space-between; align-items: center;
}
.navbar h1 { font-size: 28px; }

.cart-button {
  background: white; color: #222; border: none; padding: 10px 18px;
  border-radius: 6px; cursor: pointer; font-size: 16px;
}
.cart-button:hover { background: #ddd; }
.cart-button span {
  background: red; color: white; border-radius: 50%;
  padding: 3px 8px; margin-left: 8px;
}

main { max-width: 1200px; margin: auto; padding: 30px 20px; }
.title { text-align: center; margin-bottom: 30px; font-size: 30px; }

.product-list {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 25px;
}
.product-card {
  background: white; padding: 20px; border-radius: 10px; text-align: center;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.1); transition: 0.3s;
}
.product-card:hover { transform: translateY(-5px); }
.product-card img {
  width: 100%; height: 200px; object-fit: cover; border-radius: 8px;
}
.product-card h3 { margin: 15px 0 8px; }
.price { font-size: 20px; font-weight: bold; margin-bottom: 15px; }

.product-card button, .checkout button {
  background: #222; color: white; border: none; padding: 11px 20px;
  border-radius: 6px; cursor: pointer;
}
.product-card button:hover, .checkout button:hover { background: #444; }

.cart {
  background: white; margin-top: 40px; padding: 25px; border-radius: 10px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.1);
}
.cart h2 { margin-bottom: 20px; }
.cart-item {
  display: flex; justify-content: space-between; align-items: center;
  border-bottom: 1px solid #ddd; padding: 15px 0;
}
.cart-item h3 { margin-bottom: 8px; }
.cart-item p { margin: 5px 0; }
.remove-btn {
  background: #d9534f; color: white; border: none; padding: 8px 14px;
  border-radius: 5px; cursor: pointer;
}
.remove-btn:hover { background: #c9302c; }
.grand-total { text-align: right; margin-top: 20px; }
```

```
.checkout {
  background: white; margin-top: 40px; padding: 30px; border-radius: 10px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.1);
}
.checkout h2 { margin-bottom: 20px; }
.checkout form { display: flex; flex-direction: column; gap: 15px; }
.checkout input, .checkout textarea, .checkout select {
  width: 100%; padding: 12px; border: 1px solid #ccc;
  border-radius: 6px; font-size: 16px;
}
.checkout input:focus, .checkout textarea:focus, .checkout select:focus {
  outline: none; border-color: #222;
}
.checkout textarea { height: 100px; resize: none; }
.checkout button { width: 150px; }

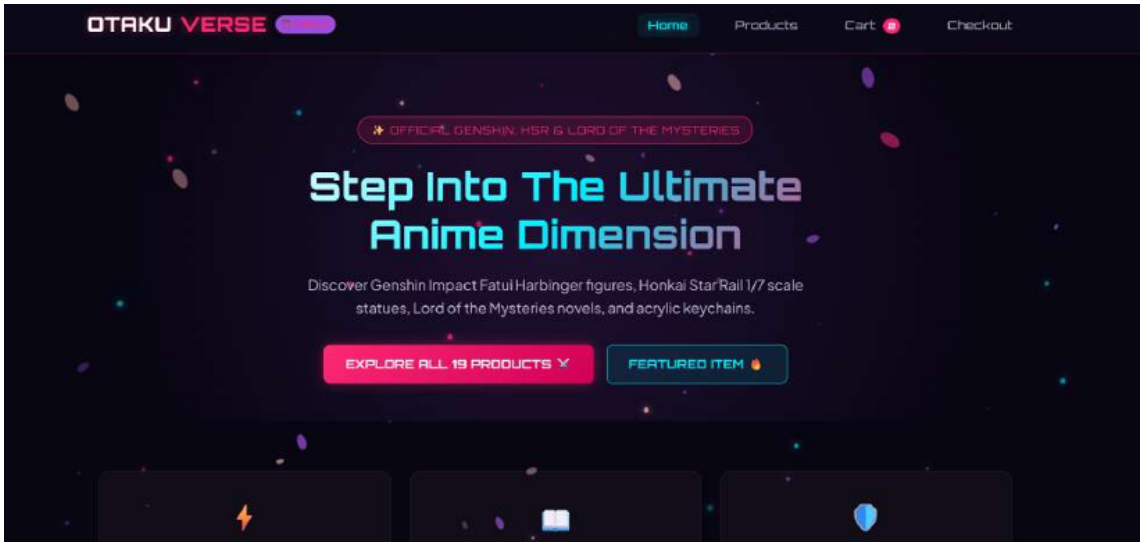
footer {
  background: #222; color: white; text-align: center;
  padding: 20px; margin-top: 50px;
}

@media (max-width: 600px) {
  .navbar { padding: 15px 20px; }
  .navbar h1 { font-size: 22px; }
  .title { font-size: 25px; }
  .cart-item { flex-direction: column; align-items: flex-start; gap: 15px; }
  .remove-btn { align-self: flex-end; }
}
```

Live Output — Screenshots

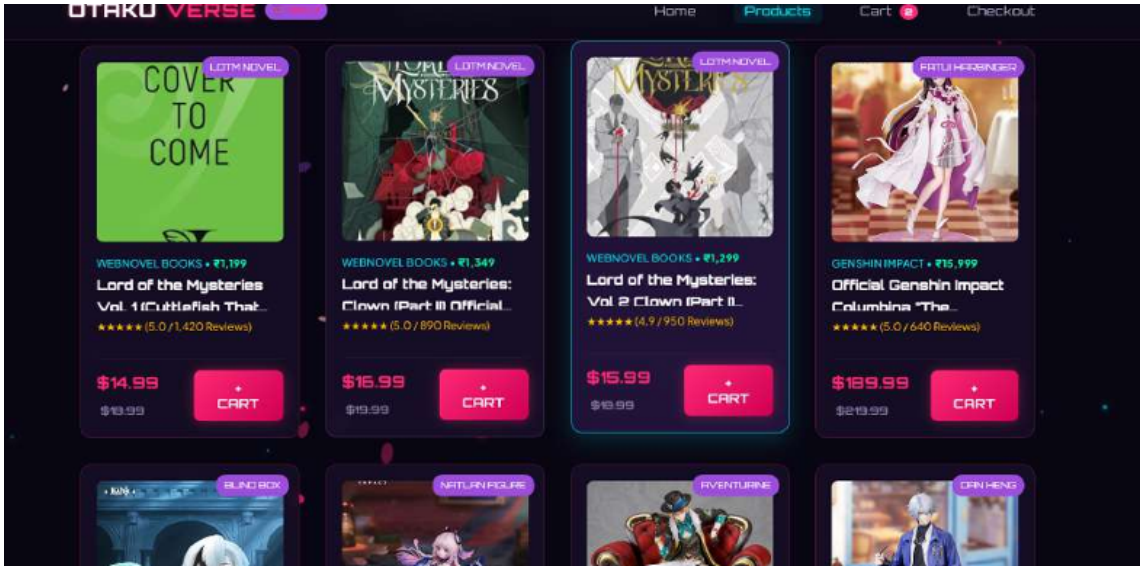
Below are screenshots of the deployed storefront (theme: OtakuVerse) demonstrating the working pages with the navbar, styling and responsive card layout applied.

1. Home Page



Header with logo & nav, hero/banner section with call-to-action buttons, built with the custom colour theme.

2. Products Page



Product listing grid — each card shows image, category badge, name, star rating, price (with strikethrough MRP) and an Add to Cart button, wrapped responsively using CSS Grid.

Full source code available at: https://github.com/cu24250059-pixel/Full_stack_BTCS303P